Encoding annotated char

Document Number:	P3263R0
Date:	2024/04/30
Reply-to:	cpp@kaotic.software
Authors:	Tiago Freire
Audience:	LWG, SG16

Abstract

This paper gives a suggestion for the creation of user annotatable character types to allow for uniform support for multiple text encodings.

Revision

#	Description
0	Initial draft

Table of Contents

1. Motivation

It is the status quo that there is no unique way of handling text in software, especially considering the range of possible applications a user might want to write. We need to acknowledge that encoding is an important consideration when handling and interpreting text, and we should make it easier on the user to do it right.
Using facilities such as the type system in order to facilitate the user to identify and manage text in different encodings is a desirable trait, and indeed distinct fundamental types such as `char8_t`/`char16_t`/`char32_t` have been added in order to facilitate partial support for Unicode (utf8/utf16/utf32).
But this solution doesn't scale, it is not practical to just add an additional fundamental char type to the core language for every encoding in existance [1], which would require not only petitioning the committee for their introduction, but also puts a burden on implementers to update secondary facilities associated with character types such as std::integral, specialization of strings and string_views, etc; and this wouldn't even support custom defined encodings and variations therof.
The ideal scenario would be to allow the users to be able to specify their own text encoding support without involvement of the standard committee.

2. What properties are desired

Before presenting the solution it is important to talk about what we want, the design should follow where we want to be first and foremost.

1. We want to be able to specify a unique type that cannot be aliased to another existing type. Solutions such as `using char_iso2022_t = char8_t` are not ideal. As the compiler would be unable to distinguish `char_iso2022_t` from `char8_t`, or objects like `std::basic_string_view<char_iso2022_t>` from

`std::basic_string_view<char8_t>`. The compiler would allow to use one in place of the other without warning, and effectively the type system would not be in use to distinguish these encodings.
2. The character should function as a character, and should allow to perform typical manipulation and transformation that one would expect to be able to do with a character. Ex. check if a code unit is a specific code unit (operator==), check if the represented code unit falls within a specific range (operators <, >, >=, etc..), allow for transformations by offset or bit manipulation (operator +, -, &, |, etc...)
3. We want to allow the user to define their own encoding easily with all the character related facilities that would be expected of it.
4. It should allow the user to specify the code unit width.
5. It should be ergonomic enough to convert.

The idea poses itself, "why not allow the user to define their own fundamental types" by copying the behavior of another fundamental type? I.e. add some form of language syntax that allows a user to define a new type name that copies the behavior of another fundamental type (such as what does `var1 + var2` mean) without it participating in overload resolution.
This idea might have its own independent merits, but the introduction of such a syntax in the core language would allow for wider range of applications that go beyond just character encodings, and as such require further analysis and raises further questions. For example, should such notation allow only for the control of overloading of a type? or should it also allow the user to tune the operator behavior (what does the operators + do)? This would have been indeed useful for things like std::byte that behaves in many ways similar to uint8_t but with the arithmetic operators taken out of it. Should this be allowed with classes? If so, what would it mean to copy the behavior of a class when that class inherits from other classes? Is it just not allowed to overload with its twin but still be able to used as an object of a base type?
This raises too many questions that would require a much closer attention to do it right, and will get in the way of getting anything practical any time soon.

Another solution that has been suggested was to use an `enum class` to define the character type. This would indeed provide a unique distinct type that cannot be overloaded with any other type.
The problem is that it would also make it an enum, making it much easier to be confused by an enum instead of an actual character type (`std::is_enum` would be true). And enum classes don't come with desired operations by default, requiring a user to re-write multiple boiler plate operators per supported encoding, making it much less ergonomic to use and not at all user friendly. Unless of course, we also provide a pre-processor macro to write all of the boiler plate for the user.

Ideally what we would want is to allow the user to declare its own unique type, allowing them to annotate specific properties that they would like this type to have, allowing for meta-programming to reflect about the properties of the type and automatically generate whatever code is needed to have this type behave like a character type as intended.

3. The proposed solution

While we were quick to dismiss adding a syntax to the language to allow for user defined types (with complexities as to be able to not only control overloads but also optionally define what operators like + mean), there is already a feature in the language that allows us to do just that. A "class".
A class is a unique type that is not overloadable with other types, you can define what operators mean, you can define the underlying storage is used, and you can even define annotations (in the form of static constexpr, and "using" directives) that allows for reflection, and you can template it allowing the user to define the minimal property set that it needs and have the remainder of the code write itself.
The proposed solution is composed of 2 components and an optional concept.

1. An encoding annotation - This uniquely identifies the encoding and is written by the user.
2. A templated character class - That takes in the encoding annotation as a template parameter and is provided by the standard

As an example, providing support for EBCDIC would be done as follows

```
//unique encoding identifier
struct text_encoding_EBCDIC: public std::text_encoding //this inheritance
annotates it as a type of encoding
{
    using char_t = char8_t;
    //static constexpr std::string_view id{"EBCDIC"}; //optional, not
required
};

using char_EBCDIC_t = std::char_enc_t<text_encoding_EBCDIC>;
```

And it would allow the user to combine it with other facilities like:

```
using string_EBCDIC = std::basic_string<char_EBCDIC_t>;
using string_view_EBCDIC = std::basic_string_view<char_EBCDIC_t>;
```

The following is an optional implementation:

```
struct text_encoding
{
    text_encoding()                     = delete; //object cannot be
instantiated
    text_encoding(text_encoding const&) = delete;
    text_encoding(text_encoding &&)     = delete;
};

template <typename T>
concept text_encoding_c =
    std::is_base_of_v<text_encoding, T>
    //&& std::same_as<std::string_view const, decltype(T::id)> //optional
    && std::is_same_v<typename T::char_t, std::remove_cvref_t<typename
T::char_t>>
    && std::unsigned_integral<typename T::char_t> && !std::is_same_v<bool,
typename T::char_t> //variant 1
    //&& (std::is_same_v<typename T::char_t, char8_t> ||
std::is_same_v<typename T::char_t, char16_t> || std::is_same_v<typename
T::char_t, char32_t>) //variant 2
    ;

template<text_encoding_c T>
class char_enc_t final
{
public:
    using encoding_t   = T;
    using underlying_t = typename T::char_t;

              inline constexpr char_enc_t(char_enc_t const&) = default;
              inline constexpr char_enc_t() = default;
    explicit inline constexpr char_enc_t(underlying_t const p_other) { _val
= p_other; }

    explicit inline           operator underlying_t& ()       { return
_val; }
    explicit inline constexpr operator underlying_t  () const { return
_val; }
              inline constexpr operator bool         () const { return
_val; }

    inline           underlying_t& value()       { return _val; }
    inline constexpr underlying_t  value() const { return _val; }

    inline char_enc_t& operator = (char_enc_t const& p_other) = default;

    inline char_enc_t& operator &= (char_enc_t const p_other) { _val &=
p_other._val; return *this; }
    inline char_enc_t& operator |= (char_enc_t const p_other) { _val |=
p_other._val; return *this; }
    inline char_enc_t& operator ^= (char_enc_t const p_other) { _val ^=
p_other._val; return *this; }
```

```
p_other._val; return *this; }
    inline char_enc_t& operator += (char_enc_t const p_other) { _val +=
p_other._val; return *this; }
    inline char_enc_t& operator -= (char_enc_t const p_other) { _val -=
p_other._val; return *this; }

    inline char_enc_t& operator <<= (uint8_t const p_offset) { _val <<=
p_offset; return *this; }
    inline char_enc_t& operator >>= (uint8_t const p_offset) { _val >>=
p_offset; return *this; }

    inline constexpr bool operator ==  (char_enc_t const p_other) const {
return _val ==  p_other._val; }
    inline constexpr bool operator !=  (char_enc_t const p_other) const {
return _val !=  p_other._val; }
    inline constexpr bool operator <   (char_enc_t const p_other) const {
return _val <   p_other._val; }
    inline constexpr bool operator >   (char_enc_t const p_other) const {
return _val >   p_other._val; }
    inline constexpr bool operator <=  (char_enc_t const p_other) const {
return _val <=  p_other._val; }
    inline constexpr bool operator >=  (char_enc_t const p_other) const {
return _val >=  p_other._val; }
    inline constexpr auto operator <=> (char_enc_t const p_other) const {
return _val <=> p_other._val; }

    inline constexpr char_enc_t operator & (char_enc_t const p_other) const
{ return _val & p_other._val; }
    inline constexpr char_enc_t operator | (char_enc_t const p_other) const
{ return _val | p_other._val; }
    inline constexpr char_enc_t operator ^ (char_enc_t const p_other) const
{ return _val ^ p_other._val; }
    inline constexpr char_enc_t operator + (char_enc_t const p_other) const
{ return _val + p_other._val; }
    inline constexpr char_enc_t operator - (char_enc_t const p_other) const
{ return _val - p_other._val; }

    inline constexpr char_enc_t operator << (uint8_t const p_offset) const
{ return _val << p_offset; }
    inline constexpr char_enc_t operator >> (uint8_t const p_offset) const
{ return _val >> p_offset; }

    inline constexpr bool       operator ! () const { return !_val; }
    inline constexpr char_enc_t operator ~ () const { return ~_val; }

    inline constexpr char_enc_t& operator ++ () { ++_val; return *this;}
    inline constexpr char_enc_t& operator -- () { --_val; return *this;}

    inline constexpr char_enc_t operator ++ (int const) { return
char_enc_t{_val++}; }
    inline constexpr char_enc_t operator -- (int const) { return
char_enc_t{_val--}; }
```

```
private:
    underlying_t _val; //intentionally not default initialized
};
```

4. Suggested properties

Many of the properties listed here are restrictive in nature, this is done to allow the character type to behave as much as possible like what would be expected from a character while reducing the potential amount of behavior that isn't "character like" and avoid unintended consequences. This shouldn't be read as if those properties are set in stone. They are a good first start, they seem sensible enough, if it turns out to be too restrictive and there is a good reason to change, then it is open too modification in the future by dropping constraints. Being too lenient might mean not being able to change them due to fears of ABI breaks.

A) It shouldn't be possible to instantiate an object of type "text_encoding" annotation. Including those created by the user.

This is an intentional design to allow for future alteration without the fear of ABI breaks. The class has no storage and should only be used as a compile time instrument to help identify the encoding and deduce its properties, such as the code unit width. Having a runtime object of such type should be ill-formed.

B) The user must inherit from `std::text_encoding` to define a text encoding.

This does two jobs.
First it signals to the compiler (by means of compile time reflection mechanism) that this type is intentionally a text encoding annotation. This prevents other types from being accidentally used.
Secondly it enforces the rule of strict compile time type that cannot be instantiated.

C) The user must define the underlying character type (minimal) which must be an unsigned fundamental integer type, other annotations are optional.

The unsigned restriction is to avoid unsavory side-effects like those that currently occur with the implementations of `char` that are signed, for example when testing if a code unit is bellow `0x32` it would unintentionally identify the code unit `0xA3` as it satisfies the criteria (i.e. `0xA3 < 0x32` is true) because `0xA3` is technically considered a negative number and negative numbers are smaller than any positive number.
Code units in character encodings are mappings, they map numbers to glyphs, they are not generic arithmetic integers and shouldn't be considered as such, having a "sign" is meaningless.
However, there are two alternatives that must be decided upon:

1. Only fundamental types that are char types can be used. i.e. `char8_t`, `char16_t`, and `char32_t`
2. Any unsigned integer type (except for bool) can be used

Option 1 would reduce the number of types in the zoo, leading to a more uniform way of handling text support facilities since it would limit design considerations to only 3 types.
Option 2 would allow for more exotic behaviors, including support for a 64bit character type, however it is doubtful if this additional versatility translates into additional utility.

D) The class `char_enc_t` should be marked as `final`, and it shouldn't be possible to inherit from it.

The same way you cannot inherit from `char` or `char8_t`, you shouldn't be able to inherit from a specialization of a `char_enc_t`. It is not clear what it would mean to have a derived character type that inherits from another base character type. They have the same encoding but are not the same character type despite the fact that they derive from the same character?

E) The following operators are defined:

The following operators were chosen because they are typically seen in character processing:
The ability to compare characters (==, <, <=, >, >=, !=, <=>).

Being able to perform code unit offsets and basic code unit offset arithmetic (+, -, ++, --).
Bit level manipulation (such as identification by bit masking, bit type transformation, etc), (&, |, ^, ~, <<, >>)
The ability to assign a new value, or modify a value in place using the previously mention operations (=, +=, -=, &=, |=, ^=, <<=, >>=)
Test if the code unit is 0 (!, bool).
Create an object from the underlying type, and the ability to cast it back to the underlying type.

F) The following operators are not defined:

The multiplication, division, and remainder (*, /, %, *=, /=, %=), have been intentionally excluded, character encodings are mappings, their intrinsic value doesn't have much numerical meaning, it is not clear what meaning multiplying (or reciprocally dividing) a character has. These are not the type of operations that I have seen associated with characters nor would I expect to be able to do them to a character type.

5. What this paper doesn't solve

This paper allows to specify a character type with an encoding annotation, which can be used with arrays, strings, and string_views, and having them function like most other characters already do, but how does one load these objects with a specific value?
While it is always possible to read and copy byte-data or prepared phrase books from an external source, in most cases developers just want the convenience of writing their strings or characters directly in the source-code using string/character literals. The problem is string/character literals are a core language feature, only "native" and utf8/utf16/utf32 encodings are supported.
Sure, it is possible to create a constexpr conversion functions to be able to translate utf8 into the user's custom encoding, and the user can and would always be expected to write their own user-defined string literals for their encoding. But how do you create your own permanent storage from within a constant evaluated context? It is not clear to me how a user would be able to do this effectively.
This paper leaves the problem of user-defined string/character literals in a constant evaluated context open, to be addressed elsewhere. It is a problem that exists, this paper provides no answer to it.

6. Questions for SG16

1. Should the underlying character type be restricted to `char8_t`, `char16_t`, `char32_t`, or should we allow for any unsigned integer?
2. Should `std::underlying_type` (currently limited to enums) be expanded to provide the underlying type of character of the code unit?
3. Should we adopt such a facility absent of a general string/character literal solution capable of outputting text with the right encoding?

7. References

1. Wiki short list of character encodings